

The Inner Compass: Context-Aware Agentic Orchestration for Enterprise IT Infrastructure

Dr. M. Sudhakar¹, Lingeswaran V², Kumaran R³, Laksheta SV⁴ and Miruthula KG⁵

Department of Computer Science and Engineering

Rajalakshmi Engineering College, Thandalam, Tamil Nadu, India

Email:¹ sudhakar.rsu@gmail.com ² 230701163@rajalakshmi.edu.in ³ 230701158@rajalakshmi.edu.in ⁴ 230701161@rajalakshmi.edu.in ⁵ 230701183@rajalakshmi.edu.in

Abstract— Enterprise IT infrastructure consists of many interconnected systems, including networks, servers, databases, containers, and virtualization platforms. Managing these systems often requires administrators to perform operations that can affect service availability and stability if performed incorrectly. In this paper, we develop and prototype a local-first supervised AI orchestration system (AI Orchestrator) to assist administrators with infrastructure management while keeping them in control of important operations. The system combines a Small Language Model (SLM), Retrieval-Augmented Generation (RAG), and a LangGraph-based agent workflow. RAG allows organization-specific information, such as network topology, configuration files, and system documentation, to be used as additional context when handling administrator requests. The agent can interact with infrastructure management tools through the LangGraph workflow, where read-only operations can be performed automatically while configuration-changing operations require explicit administrator approval. The system also uses service credentials stored by the admin for infrastructure access without directly exposing infrastructure credentials to the AI agent. The prototype show how SLM can assist with infrastructure management and human oversight over potentially disruptive actions.

Keywords— *Small Language Model, Retrieval-Augmented Generation, LangGraph, Model Context Protocol, IT Infrastructure Management.*

I. INTRODUCTION

Modern organizations rely on complex IT infrastructures consisting of interconnected networks, servers, databases, containers, virtualization platforms, and vendor-specific technologies. Administrators must configure, monitor, troubleshoot, and maintain these systems using various tools. As infrastructure becomes larger and more interconnected, a single incorrect operation can affect connectivity, service performance, or system stability. Human error is therefore an important concern in infrastructure administration, with

previous studies identifying it as a significant contributor to system outages.

The impact of such mistakes can be particularly significant in large-scale cloud environments. A recent Azure service incident in October 2025 demonstrated how an operational change involving Azure Front Door could lead to connectivity and performance problems across multiple services. Such incidents highlight the need for safer administration approaches that reduce the risk of incorrect operations while keeping administrators in control.

Large Language Models (LLMs) and AI agents provide new opportunities to assist administrators by interpreting requests, retrieving relevant information, and interacting with infrastructure tools. However, allowing an AI system to directly perform infrastructure operations introduces risks such as incorrect interpretation, unintended configuration changes, and exposure of sensitive information. Cloud-based AI services may also be unsuitable for organizations that require their infrastructure data to remain within their own environment or need operation without continuous Internet connectivity.

This paper presents a local-first, supervised AI orchestration system for infrastructure management. The system combines a Small Language Model (SLM), Retrieval-Augmented Generation (RAG), and a LangGraph-based agent workflow. RAG provides organization-specific information, such as network topology, configuration files, and system documentation, as additional context to the agent. LangGraph manages interactions between the agent and infrastructure tools. Read-only operations can be performed automatically, while configuration-changing operations require explicit administrator approval. The system also uses stored service credentials without directly exposing privileged credentials to the AI agent. MCP-based integration of organization-specific and vendor-specific tools is considered as future work.

II. LITERATURE REVIEW

Traditional infrastructure management has primarily relied on manual administration and rule-based automation using

command-line tools, scripts, monitoring systems, and configuration-management tools. While these approaches provide predictable behavior, they require significant domain knowledge and separately defined procedures for different technologies, making them difficult to maintain as enterprise infrastructures become more heterogeneous.

Machine-learning-based AIOps approaches addressed some of these limitations by using infrastructure logs, metrics, and events for anomaly detection, failure prediction, incident identification, and resource optimization. However, these approaches mainly focus on monitoring and prediction rather than understanding natural-language requests and performing multi-step infrastructure operations.

Large Language Models (LLMs) introduced natural-language interaction and agent-based task execution, allowing systems to interpret requests, reason over information, select tools, and perform multi-step operations. However, large models often depend on cloud infrastructure and require substantial computational resources, raising concerns about privacy, latency, cost, and offline operation. Smaller Language Models (SLMs) provide a more practical option for local deployment, but their limited knowledge of organization-specific and current infrastructure information creates a need for external knowledge sources.

Retrieval-Augmented Generation (RAG) addresses this limitation by providing models with relevant information from external sources such as system documentation, network topology, configuration files, and policies. Agent orchestration frameworks such as LangGraph can then coordinate the model, retrieved information, and infrastructure tools across multiple steps. However, giving an AI agent direct access to administrative tools introduces the risk of unintended or unsafe operations.

Human-in-the-loop (HITL) approaches provide an additional safety mechanism by requiring administrator approval for potentially high-risk operations while allowing low-risk or read-only operations to be automated. The Model Context Protocol (MCP) also provides a standardized approach for connecting AI agents with organization-specific and vendor-specific tools, although it primarily addresses tool interoperability rather than execution safety.

Existing research therefore addresses different aspects of AI-assisted infrastructure management, including prediction, natural-language interaction, local models, knowledge retrieval, tool orchestration, and human supervision. However, combining these capabilities into a local-first and supervised system for heterogeneous enterprise infrastructure remains less explored. This work addresses this gap by combining SLM-based reasoning, RAG-based organizational context,

LangGraph-based orchestration, and risk-based human approval within a single infrastructure management framework. MCP-based tool integration is considered as future work.

III. PROPOSED SOLUTION

The proposed framework, SLOps, is a local-first, AI-driven system designed to manage enterprise IT infrastructure securely. It addresses common issues found in cloud-based AI tools, such as the need for constant internet connectivity, the risk of exposing credentials, and loss of context between operations. The architecture separates the reasoning (“thinking”) of the system from its execution (“doing”) across five main components.

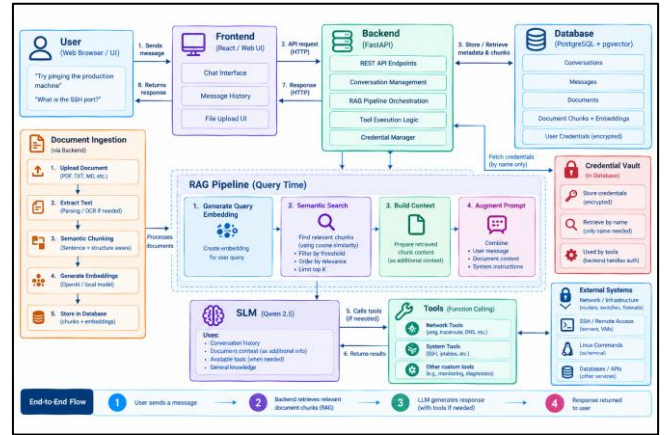


Fig. 1. System architecture of the proposed framework.

A. API and Conversation Management

Administrators interact with the system through an API. Each request belongs to a conversation, allowing previous interactions to be maintained. LangGraph checkpointing preserves the workflow state, allowing interrupted tasks to resume after events such as administrator approval.

B. Local SLM Agent

A local Small Language Model acts as the reasoning component. It interprets administrator requests, uses available contextual information, and determines which infrastructure tools are required. Keeping the model local reduces the need to send sensitive infrastructure information to external AI services.

C. Organization-Specific Knowledge Retrieval

The system uses Retrieval-Augmented Generation (RAG) to provide organization-specific context from documents such as configuration files, network information, and system documentation. Documents are divided into sentence-based or bounded character chunks, converted into embeddings, and stored in a vector database. Relevant information is retrieved and provided to the agent when processing a request.

D. LangGraph Workflow

LangGraph manages the interaction between the SLM, tools, and approval process. It maintains the workflow state and coordinates multiple reasoning and tool-execution steps. Checkpointing allows the workflow to be paused and resumed without losing its state.

E. Infrastructure Tool Execution

The system provides tools for infrastructure operations such as network diagnostics and firewall management. Tools are categorized based on their operation type. Read-only tools can be selected and executed automatically, while tools that modify infrastructure require administrator approval.

F. Credential and SSH Management

Remote operations are performed through SSH using credentials stored and managed by the system. Administrators do not need to provide privileged credentials in their requests, and the AI agent interacts with the SSH functionality through tools rather than directly accessing stored credentials.

G. Human-in-the-Loop Control

Potentially disruptive operations, such as modifying firewall rules, require explicit administrator approval. The workflow pauses before execution and presents the proposed operation to the administrator. The operation proceeds only after approval; if rejected, the decision is returned to the agent so it can respond appropriately.

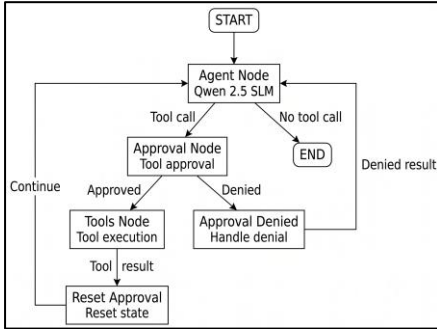


Fig. 2. Agent workflow and human-in-the-loop approval process.

IV. SYSTEM IMPLEMENTATION

To realize the architecture outlined in Section III, the framework is built using a modern Python-based technology stack. The system strictly separates concerns across the API, agent orchestration, model reasoning, tool execution, and knowledge retrieval layers. A core design principle is the use of a tri-database architecture, in which application data, agent state memory, and vector embeddings are independently isolated.

A. API Gateway and Application Database

The primary interface is a RESTful API developed using the FastAPI framework. This layer handles incoming natural-language queries from administrators and manages user sessions. Application-level data is handled by the first of the framework's three databases: a dedicated Application Database (e.g., PostgreSQL). This relational database handles all standard CRUD (Create, Read, Update, Delete) operations and stores user credentials, session details, and role-based access control (RBAC) configurations. Isolating this database ensures that the AI agent's operations do not interfere with core user authentication and application security.

B. Stateful Orchestration and Agent Memory

Once a user request is authenticated, the FastAPI server forwards the input to the orchestration module, built using LangGraph. LangGraph structures the agent's workflow as a directed graph, managing the cyclical transitions between planning, tool execution, and human-in-the-loop pauses. To maintain the agent's memory, the system implements a second database, the Orchestration Database, managed by LangGraph's native checkpointing system. This database automatically saves the agent's exact state, conversation history, and intermediate command outputs at every step. If the agent pauses to wait for administrator approval of a firewall change, the session state is safely frozen and can be resumed once approval is granted.

C. Local Cognitive Engine and LoRA Fine-Tuning

The central reasoning and decision-making engine is the Qwen2.5-7B Small Language Model (SLM). While standard open-source models possess general reasoning capabilities, managing specific IT infrastructure requires deep, domain-specific accuracy. To achieve this, the Qwen2.5-7B model is fine-tuned using Low-Rank Adaptation (LoRA). The fine-tuning process trains the model on a curated dataset comprising network diagnostic logs, infrastructure configurations, and general organizational operational data. This parameter-efficient fine-tuning allows the SLM to accurately interpret complex network states and autonomously select the correct sequence of tools for a given task, ensuring high accuracy without the computational overhead of full-parameter training.

D. Secure Tool Execution and Modularity

When the fine-tuned Qwen2.5-7B SLM selects a tool to fulfill an objective, the framework executes the command using secure Python libraries. Local host diagnostics and basic network commands are handled via restricted subprocess execution. For remote infrastructure management, the system uses the Python paramiko library, enabling the agent to programmatically establish and manage secure SSH sessions on target servers. The execution layer validates each requested operation before invoking the selected tool, reducing

unintended infrastructure changes while maintaining administrator oversight for sensitive operations.

E. Knowledge Retrieval and Vector Database

The system implements Retrieval-Augmented Generation (RAG) using PostgreSQL with the pgvector extension. Uploaded documents are divided into smaller chunks using sentence-based or bounded character-based chunking, and each chunk is converted into an embedding using the BAAI/bge-small-en-v1.5 embedding model. The generated embeddings and corresponding document chunks are stored in the PostgreSQL vector database. When an administrator submits a request, the query is converted into an embedding and compared with stored embeddings using cosine similarity. Relevant chunks are retrieved and provided as additional context to the agent during request processing.

F. Human-in-the-Loop Implementation

The system implements HITL control for state-changing infrastructure operations. Read-only tools are executed automatically, while operations such as firewall modifications require administrator approval before execution. When approval is required, the LangGraph workflow pauses and presents the proposed operation to the administrator. The workflow resumes after approval, while a rejection is returned to the agent so it can respond appropriately without executing the operation. Checkpointing preserves the workflow state during this process.

V. EXPERIMENTAL SETUP AND EVALUATION

To validate the feasibility, accuracy, and security of the proposed framework, a containerized testbed environment was established. The primary objective of this evaluation is to assess the local SLM's ability to interpret administrative intent, execute diagnostic and state-mutating commands securely, and maintain statefulness without relying on external cloud APIs.

A. System Requirements and Hardware Specifications

Because the system runs the entire orchestration pipeline and the LLM reasoning engine locally, specific hardware requirements must be met to ensure acceptable inference latency. The experimental host machine was configured with a modern multi-core CPU (e.g., Intel Core i7 / AMD Ryzen 7 or higher) to handle Docker bridge routing, database operations, and concurrent LangGraph state management. A minimum of 16 GB of system RAM is required to accommodate the Docker container overhead, PostgreSQL databases, and the memory footprint of the Qwen2.5-7B model fine-tuned via LoRA, which typically requires approximately 6–8 GB of memory when using 4-bit or 8-bit quantization. The software stack runs on a standard Linux distribution using Docker Engine and

Docker Compose for container orchestration, and Python 3.10+ for the LangGraph and FastAPI backend.

B. Testbed Environment and Topology

The testing environment was constructed using Docker Compose to simulate an enterprise local area network (LAN). The topology consists of two primary containers running in an isolated environment: a Management Server Node, which hosts the core framework services, including the FastAPI application gateway, the LangGraph orchestration engine, the PostgreSQL checkpointing store, and the locally fine-tuned Qwen2.5-7B SLM; and a Target Node, which simulates an enterprise Linux server requiring administrative management and is pre-configured with network utilities, firewall tools (e.g., iptables or ufw), and a running OpenSSH daemon. The containers communicate over a custom Docker bridge network, which provides dedicated virtual IP addresses and internal DNS resolution, closely mimicking standard enterprise subnet routing without exposing the testbed to external networks.

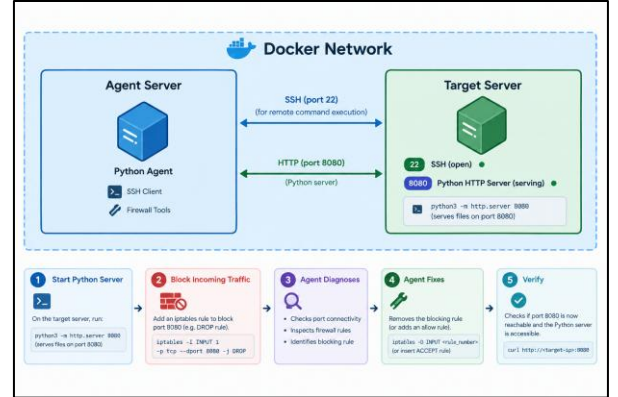


Fig. 3. Test Docker Environment with Test Containers.

C. Authentication and Privilege Scoping

Remote management operations are conducted over SSH using the Python paramiko library. To enable non-interactive command execution by the agent, a dedicated service account is provisioned on the target node exclusively for automated orchestration tasks. Administrative tasks, such as modifying firewall rules, require root privileges; the target node's sudoers configuration grants the service account non-interactive execution rights (NOPASSWD) strictly for designated system-administration binaries. In a production deployment, this configuration does not introduce a security vulnerability, since network and systems administrators can strictly govern the service account using existing Role-Based Access Control (RBAC) or Identity and Access Management (IAM) frameworks. By restricting the account's permissions to only the necessary diagnostic and management binaries, the deployment adheres to the principle of least privilege.

D. Evaluation Metrics and Test Scenarios

The framework was evaluated against common IT operational workflows using two primary parameters: Task Success Rate, the ability to correctly choose a tool and execute the corresponding command, and Execution Autonomy, the ability to correctly differentiate between safe automated tasks and tasks requiring human approval. Table I summarizes the categories of operations exercised during testing.

TABLE I
TESTED OPERATIONAL CATEGORIES AND OUTCOMES

Category	Target Command / Action	Execution Mode	Result
Network Diagnostics	ping -c 4 <target_ip>	Automated (Subprocess/SSH)	Success
DNS Resolution	dig <hostname>	Automated (Subprocess/SSH)	Success
Interface Inspection	ip addr show	Automated (SSH)	Success
Firewall Management	Port blocking / unblocking	Human-Approved (SSH)	Success
ICMP Traffic Control	Block / allow ICMP packets	Human-Approved (SSH)	Success

VI. RESULTS AND DISCUSSION

The experimental evaluation demonstrates that the proposed system can combine local AI reasoning, organization-specific knowledge, infrastructure tools, and administrator approval within a single workflow. The results are discussed in terms of tool execution, stateful orchestration, error handling, and operational safety.

A. Tool Execution and Orchestration

The system successfully performed both local and remote infrastructure operations using the implemented tool layer. Local operations were executed through subprocesses, while remote operations were performed through SSH using Paramiko. For multi-step requests, the agent was able to use the output of one tool as context for subsequent operations, allowing related diagnostic and configuration tasks to be handled within the same workflow.

B. Stateful Workflow Management

LangGraph checkpointing allowed the system to maintain conversation and workflow state across multiple tool interactions. When an operation required administrator approval, the workflow could be paused and resumed without losing the existing context. This was particularly useful for workflows involving multiple diagnostic and configuration steps.

C. Error Handling

The execution layer captured command output, error output, return codes, and connection failures. When errors such as unreachable hosts or insufficient permissions were encountered,

the resulting information was returned to the agent instead of being treated as a successful operation. This allowed the system to provide the administrator with the actual execution result and corresponding error information.

D. Human-in-the-Loop Safety

The evaluation showed that state-changing infrastructure operations were stopped before execution until administrator approval was received. Read-only operations could proceed without approval, while operations such as firewall modifications required explicit confirmation. This provides a separation between automated reasoning and potentially disruptive infrastructure changes.

E. Discussion

The results indicate that the combination of an SLM, RAG, LangGraph, infrastructure tools, and HITL can provide a practical approach to AI-assisted infrastructure management. RAG allows the agent to use organization-specific information without modifying the model, while the tool layer restricts infrastructure interaction to predefined operations. The HITL mechanism adds an additional control point for state-changing actions. However, the current evaluation is based on a prototype and a limited set of infrastructure operations. Further testing with larger workloads, concurrent users, additional infrastructure technologies, and quantitative performance measurements is required to evaluate scalability and reliability in larger environments.

VII. FUTURE SCOPE

Future work will focus on extending the framework from a single-agent orchestrator to a Domain-Specific Multi-Agent Architecture. As enterprise IT environments grow in complexity, a single agent may face increasing context and task-management challenges. Using LangGraph's multi-agent capabilities, future versions could include specialized agents for domains such as network management, Linux systems, database administration, and Kubernetes.

Three main directions are planned. Under Domain-Specific State Management, each sub-agent could maintain its own relevant state within the PostgreSQL database, reducing unnecessary cross-domain context. Under Modular Tool Integration, the system would add the Model Context Protocol (MCP) to provide a standardized mechanism for connecting domain-specific and vendor-specific tools to the agents. Under Hierarchical Routing, a lightweight supervisor agent could receive the administrator's request, divide it into domain-specific tasks, and delegate them to the appropriate specialized agents.

VIII. CONCLUSION

This paper presented a local-first supervised AI orchestration system for assisting administrators with enterprise IT infrastructure management. The proposed system combines a Small Language Model, Retrieval-Augmented Generation, LangGraph-based workflow orchestration, infrastructure tools, credential management, and human-in-the-loop control. Organization-specific information can be retrieved from PostgreSQL with pgvector and provided as additional context, while infrastructure operations are performed through predefined tools.

The experimental evaluation demonstrated that the system can perform local and remote infrastructure operations, maintain workflow state across multiple tool interactions, handle execution errors, and require administrator approval before state-changing operations. The use of checkpointing also allows interrupted workflows to resume while preserving their existing state. These results demonstrate the feasibility of combining local AI reasoning with controlled infrastructure tool execution while keeping administrators involved in potentially disruptive operations.

Future work will extend the system with domain-specific multi-agent capabilities and Model Context Protocol (MCP) support for modular tool integration. These extensions can allow the framework to support a wider range of infrastructure domains while maintaining the existing principles of local operation, controlled tool access, and administrator supervision.

REFERENCES

[INFORMATION NOT PROVIDED — no reference list was included in the source document. Insert the project's citation list here before submission.]